

Ex07 — v-if / v-show : Statut de match

Informations

Notion	Rendu conditionnel — <code>v-if</code> , <code>v-else</code> , <code>v-show</code>
Niveau	● Facile
Durée estimée	15 minutes
Travail	Individuel — sans aide IA

Objectif

Rendre dynamique l'affichage du statut d'un match sportif en utilisant `v-if`, `v-else` et `v-show`.

Le HTML et le CSS sont fournis — vous ne devez écrire que la **logique Vue.js**.

Structure de l'exercice

```
/ex07-statut-match
├── src/
│   └── App.vue
└── index.html
```

Résultat attendu



Ce qui est fourni

Copiez ce code dans `App.vue` — le HTML et le CSS sont déjà en place. Vous n'avez qu'à compléter le `<script setup>` et modifier le `<template>`.

```
<script setup>
// À compléter
</script>

<template>
  <div class="card">
    <div class="status-live">Match en cours</div>
    <div class="status-ended">● Match terminé</div>

    <div class="match">
      <div class="team">team1</div>
      <div class="score">score1 - score2</div>
      <div class="team">team2</div>
    </div>

    <p class="time">🕒 elapsedTime' écoulées</p>

    <div class="result">
      🏆 Vainqueur : winner
    </div>
  </div>
</template>

<style scoped>
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Arial', sans-serif;
  background: #0f172a;
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
}

.card {
  background: #1e293b;
  border-radius: 16px;
  padding: 2rem;
  min-width: 380px;
  box-shadow: 0 8px 32px rgba(0, 0, 0, 0.4);
  text-align: center;
}

.status-live {
  display: inline-flex;
  align-items: center;
  gap: 0.4rem;
}
```

```
background: #dcfce7;
color: #166534;
border-radius: 999px;
padding: 0.3rem 1rem;
font-size: 0.85rem;
font-weight: bold;
margin-bottom: 1.5rem;
}

.status-live::before {
  content: '';
  width: 8px;
  height: 8px;
  background: #16a34a;
  border-radius: 50%;
  animation: pulse 1s infinite;
}

.status-ended {
  display: inline-block;
  background: #374151;
  color: #9ca3af;
  border-radius: 999px;
  padding: 0.3rem 1rem;
  font-size: 0.85rem;
  font-weight: bold;
  margin-bottom: 1.5rem;
}

@keyframes pulse {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.3; }
}

.match {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 1.5rem;
  margin-bottom: 1.5rem;
}

.team {
  font-size: 1rem;
  font-weight: bold;
  color: #e2e8f0;
  min-width: 100px;
}

.score {
  font-size: 2.5rem;
  font-weight: bold;
  color: #f1f5f9;
  letter-spacing: 0.1em;
}
```

```
}

.time {
  color: #94a3b8;
  font-size: 0.9rem;
  margin-bottom: 0.5rem;
}

.result {
  background: #1d4ed8;
  color: #bfdbfe;
  border-radius: 8px;
  padding: 0.75rem 1.5rem;
  font-size: 0.95rem;
  font-weight: bold;
  margin-top: 0.5rem;
  display: inline-block;
}
</style>
```

À faire

- ☒ Définir les données réactives avec `ref()` :
 - `team1`, `team2` — noms des équipes
 - `score1`, `score2` — scores
 - `isLive` — booléen indiquant si le match est en cours
 - `elapsedTime` — temps écoulé en minutes
 - `winner` — nom de l'équipe gagnante
- ☒ Remplacer les textes en dur par `{{ }}`
- ☒ Afficher le badge **"Match en cours"** avec `v-if` si `isLive` est `true`
- ☒ Afficher le badge **"Match terminé"** avec `v-else`
- ☒ Afficher le temps écoulé avec `v-show` uniquement si le match est en cours
- ☒ Afficher le résultat final avec `v-if` uniquement si le match est terminé
- ☒ Pas d'utilisation d'IA pour générer le code

💡 **Test** : Changez `isLive` de `true` à `false` et vérifiez que le badge, le temps écoulé et le résultat final s'affichent correctement.